INSTRUCTIONS :

1. Complete all questions in your designated project group.
2. All members must contribute to writing the codes. (i.e. 1 question = 1 person, and share the workload if there's an additional question relative to the actual number of members in your team (i.e. 5)). Ensure that all members must understand and explain codes from any of the questions.
3. During viva, all students in each team will be randomly asked to describe, answer and edit any of the answers provided. Marks will be given to your ability to present the answers.

Lab Report

Prepare a report to solve the above problems. The report should contain all the sections as below for each question:

Section	Description
1. Problem	Description on the problem
2. Solution	Explanation on how to solve the problems below
3. Sample Input & Output	A few sets of input and output (snapshot)
4. Source code	Java source code

Requirements

1. Group Assignment (Grouping is the same as your project group)
2. Cover page that includes all student matric number and full name.
3. Font: Times New Roman 12, Line Spacing: 1 ½ Spacing
4. Submit to Spectrum according to your OCC. **Deadline** : Before your viva session (W6).

Question 1: Friend or Foe?

Problem Statement:

In the fantasy world of Ravnora, usernames are not just identifiers, but they also hint at the nature of their owner. A powerful wizard named Eldranar has devised a method to determine if a username belongs to a friend or a foe based on the distinct magical runes in it.

Eldranar's rule is simple: if the number of distinct runes (characters) in a username is odd, then the user is considered a foe, otherwise a friend.

You are given a string that represents a username, consisting of lowercase English runes (letters). Help Eldranar determine if this user is a friend or a foe based on his method.

Input:

The first line contains a non-empty string that contains only lowercase English runes (letters) — the username. This string contains at most 100 runes.

Output:
If the username corresponds to a friend, print "ALLY DETECTED!" (without the quotes), otherwise, print "ENEMY ALERT!" (without the quotes).

Sample Output:

Input	Output
Elvanar	ALLY DETECTED!
gorgoth	ENEMY ALERT!
arcanor	ALLY DETECTED!

Tips:
- For the first example: There are 4 distinct runes in "elvanar". These runes are: "e", "l", "v", "a", "n", "r". So elvanar corresponds to a friend and you should print "ALLY DETECTED!".
- For the second example: There are 7 distinct runes in "gorgoth". These runes are: "g", "o", "r", "t", "h". Thus, gorgoth is considered an enemy, and you should print "ENEMY ALERT!".

Question 2 : School Game Day Coding Challenge

Problem Statement

You are a teacher at a school that is holding a coding challenge as part of its annual Game Day event. The challenge for the participants is to create a program that simulates a classic game that reinforces number skills and logical thinking. The game is called **"LuluLala"**.

In this game, participants have to write a program that asks user to enter a range of number (such as 1 as start and 100 as end) and prints numbers from 1 to 100. However, the rules for the game are as follows:

- If a number is a multiple of 3, the program should print "Lulu" instead of the number.
- If a number is a multiple of 5, the program should print "Lala" instead of the number.
- If a number is a multiple of both 3 and 5, the program should print "LuluLala".
- For all other numbers, the program should print the number itself.

Input Format :

The program should prompt the user to enter two integers representing the start and end of a series of numbers.

Sample output:

```
Enter the starting value : 1
Enter the ending value : 100

The output is :
1
2
Lulu
4
Lala
Lulu
7
8
Lulu
Lala
11
Lulu
13
14
LuluLala
16
…
```

Question 3: Defining a Valid Triangle

Problem Statement:
A triangle is defined by three angles. For a triangle to be valid:

1. The sum of all three angles must equal 180°.
2. Each angle must be greater than 0°.

Write a Java program that takes three angles as input and determines if the triangle is valid or not based on the conditions mentioned. Determines the type of triangle based on the angles if the triangle is valid

Hint:
The sum of the three angles of a triangle is equal to 180°.

Triangle Type Rules:

1. **Equilateral Triangle**: All three angles are equal.
2. **Isosceles Triangle**: Any two angles are equal.
3. **Scalene Triangle**: All three angles are different.
4. **Right-Angled Triangle**: One of the angles is exactly 90°.

Input Format:
The program should prompt the user to enter three integers representing the angles of a triangle.

Sample Output:

Input	Output
60 60 60	The triangle is valid. It is an equilateral triangle.
90 45 45	The triangle is valid. It is a right-angled triangle. It is an isosceles triangle.
100 60 40	The triangle is not valid.

Question 4: Input validation

Problem statement:

Input validation is a simple yet very important part of programming. Write a program that asks the user to input a positive integer. If the user does not comply (by inputting text, floating values, or negative integers), inform the user that their input is invalid, and re-prompt the user to input a *valid* positive integer until the user complies. When a valid integer is inputted, declare if it is even or odd.

Hint: you may use input.charAt(i) when looping through the inputted text, where "input" is the variable where the input is stored.

Sample output:

```
run:
Please input a positive integer: fourteen
Invalid input. Please re-input a valid +ve integer: 14.5
Invalid input. Please re-input a valid +ve integer: -14
Invalid input. Please re-input a valid +ve integer: 14
you've inputed a valid integer!
The integer is even!
```

Question 5: Digit Grouping

Problem Statement:
You are given a positive integer n. Your task is to group the digits of nnn into groups such that each group contains the same digit repeatedly, and each digit group is as long as possible. The digit groups should be in the order of appearance in nnn.

For example, if n=1223334444, you can split it into "1", "22", "333", "4444".
Once the digit groups are formed, your task is to determine:

1. The total number of digit groups.
2. The digit that forms the longest group and its length.
3. The sum of lengths of all digit groups.

If there is a tie for the longest group (multiple groups have the same length), choose the digit that appears first.

Input:
The input consists of a single positive integer nnn (1 ≤ n ≤ 10^18). The number n is guaranteed to have no leading zeros.

Output:
Print three lines:

1. The total number of digit groups.
2. The digit forming the longest group and its length.
3. The sum of the lengths of all digit groups.

Sample Output:

Input	Output
1223334444	4 4 4 10
11122221111	3 2 4 11

Explanation:

- Groups are: "1", "22", "333", "4444".
  - Total number of groups: 4.
  - The longest group is "4444" with length 4.
  - The sum of lengths of all groups: 1 + 2 + 3 + 4 = 10.

- Groups are: "111", "2222", "1111".
  - Total number of groups: 3.
  - The longest group is "2222" with length 4 (appears before "1111" with the same length).
  - The sum of lengths of all groups: 3 + 4 + 4 = 11.

Question 6: Pattern Printing

Problem Statement:
Write a Java program that prints the word "**MALAYSIA**" in a specific star **(*)** pattern format. Each letter should be represented in a **7x7** grid, following the star patterns shown below.

Pattern Specifications:

1. Each letter in "**MALAYSIA**" is printed in a **7x7** grid.
2. Use * characters to form each letter, with appropriate spacing between the letters.
3. Use loops and conditional statements to construct each letter's pattern within the main loop.
4. Print the letters horizontally with spaces between them to keep the word aligned.

Tips:

1. Use a single for loop to iterate through each row (from 0 to 6), then use nested for loops and if statements to define the conditions for each letter.
2. Add **System.out.print(" ");** between each letter to create spacing for better readability.
3. Think of each letter as a grid and use row and col values to define where to place * and within the grid.
4. At the end of each row, use **System.out.println();** to move to the next line, keeping the 7x7 grid format for each letter.

Sample Output:

```
run:
*     *   *******   *         *******   *     *   *******   *******   *******
**   **   *     *   *         *     *   *   *     *           *     *     *
* * * *   *     *   *         *     *   * *     *           *     *     *
*  *  *   *******   *         *******     *     *******     *     *******
*     *   *     *   *         *     *     *           *     *     *     *
*     *   *     *   *         *     *     *           *     *     *     *
*     *   *     *   *******   *     *     *     *******   *******   *     *
BUILD SUCCESSFUL (total time: 0 seconds)
```